

SERVERLESS IMAGE/VIDEO PROCESSING

Services: S3, Lambda, Step Functions, Rekognition, SNS

Goal: Automatically process images or videos uploaded to S3 (e.g., resize, label detection).

Skills: Event-driven architecture, serverless orchestration.

Definition

This project implements a fully automated, event-driven serverless architecture on AWS that processes images uploaded to Amazon S3. When a new image is uploaded, an AWS Step Functions workflow is triggered automatically. The workflow orchestrates multiple AWS Lambda functions to perform image pre-processing, detect labels using Amazon Rekognition, and send notifications via Amazon SNS.

The architecture is fully serverless, scalable, and does not require any server management.

Scope of the Project

The scope of this project includes:

Automatic image processing on upload

AI-based label detection using Amazon Rekognition

Workflow orchestration using AWS Step Functions

Event-driven automation using S3 triggers

Email notification system using Amazon SNS

Implementation of secure IAM roles and permissions

Future scope:

Video processing support

Face recognition integration

Error handling and retry mechanisms

Integration with API Gateway

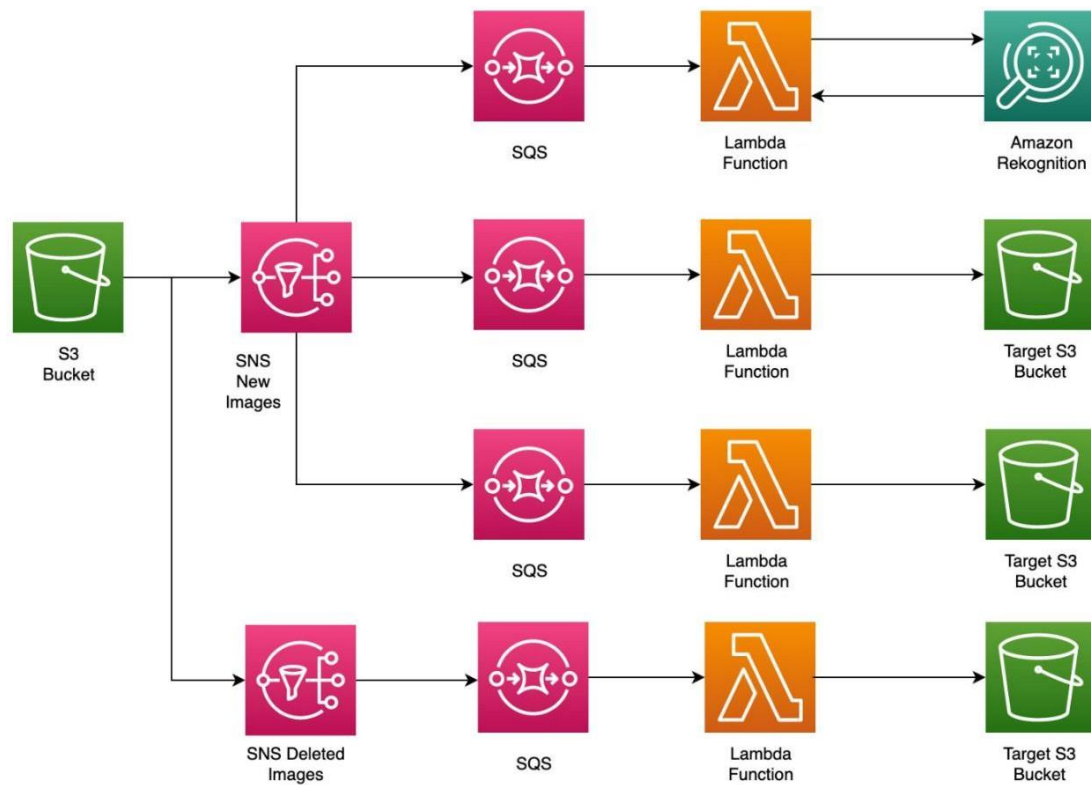
Storage of results in DynamoDB

CI/CD automation for deployment

Methodology

The project follows an event-driven serverless architecture methodology.

Architecture Flow:



Key Design Principles:

Serverless computing (no EC2 usage)

Event-driven automation

Microservices-based Lambda functions

Workflow orchestration using Step Functions

Least privilege IAM security model

Modular and scalable architecture

Explanation of Services Used

◆ Amazon S3

Purpose:

Stores input images

Generates event notification on object upload

Acts as workflow trigger

◆ AWS Lambda

Purpose:

Executes backend logic without managing servers

ResizeImage Lambda: Pre-processing of image

DetectLabels Lambda: Calls Rekognition API

SendNotification Lambda: Publishes SNS notification

Trigger Lambda: Starts Step Function execution

◆ AWS Step Functions

Purpose:

Orchestrates multiple Lambda functions

Manages workflow sequence

Handles state transitions

Provides execution monitoring

◆ Amazon Rekognition

Purpose:

Performs AI-based label detection

Identifies objects within the image

Returns detected labels with confidence scores

◆ Amazon SNS

Purpose:

Sends email notifications

Publishes detected labels to subscribers

◆ IAM (Identity and Access Management)

Purpose:

Manages roles and permissions

Secures cross-service communication

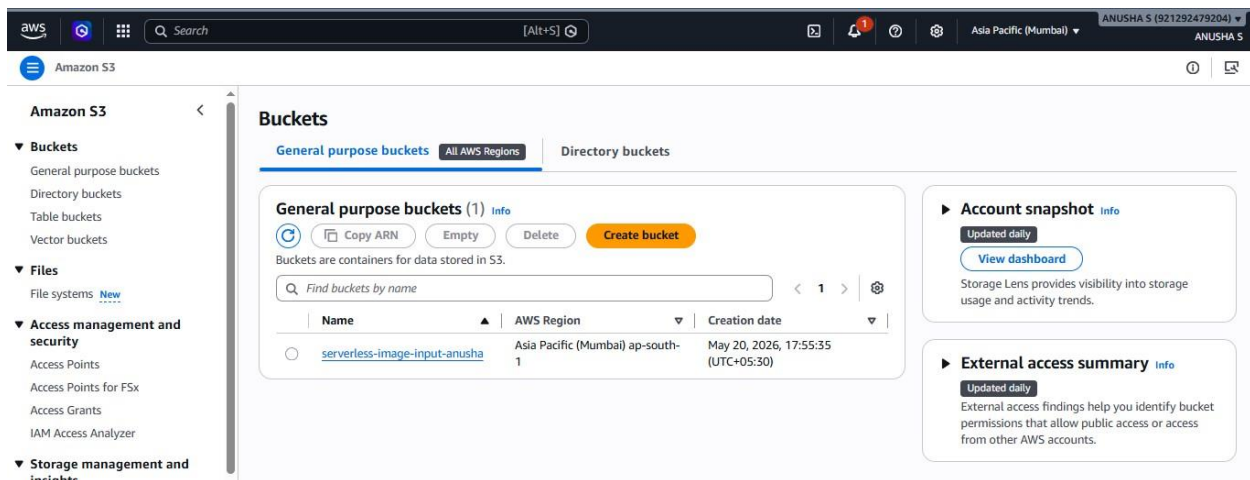
Implements least privilege principle

Step-by-Step Procedure

Step 1: Create S3 Bucket

Created input bucket: serverless-image-input-megha

Configured bucket for object creation events



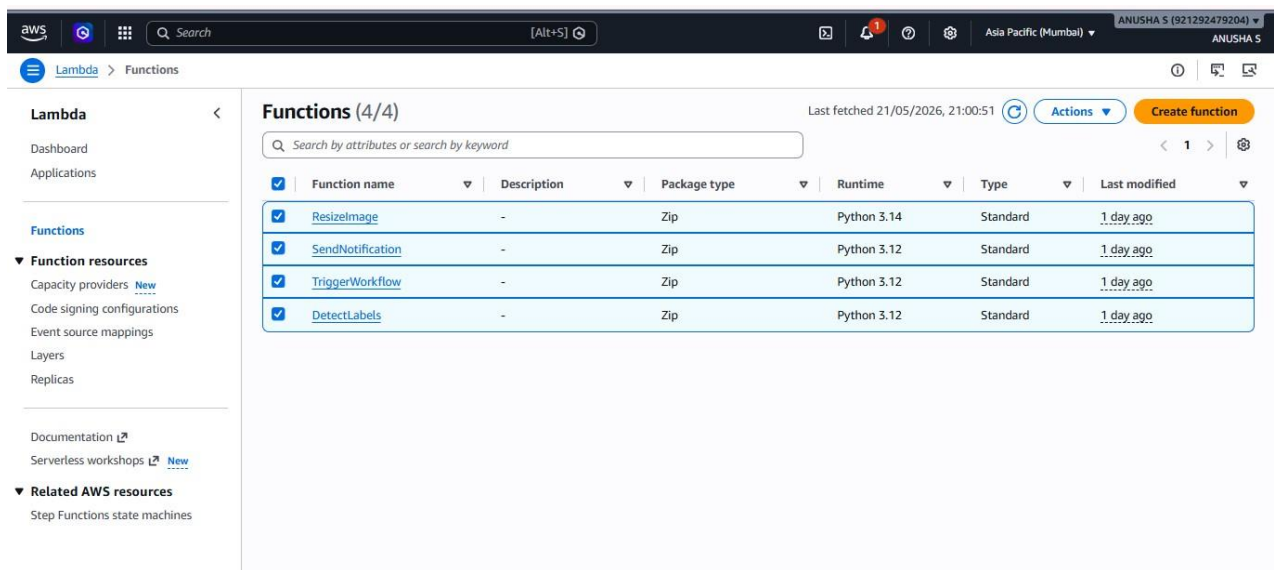
Step 2: Create Lambda Functions

ResizeImage Lambda

Accepts bucket and key as input

Performs pre-processing

Returns bucket and key for next state
 DetectLabels Lambda
 Uses boto3 Rekognition client
 Calls detect_labels API
 Returns detected labels
 SendNotification Lambda
 Uses SNS client
 Publishes label results to SNS topic
 Trigger Lambda
 Triggered by S3 event
 Starts Step Function execution
 Passes bucket and key as input



The screenshot shows the AWS Lambda console interface. On the left is a navigation sidebar with 'Lambda' selected, containing links to 'Dashboard', 'Applications', 'Functions', 'Function resources', and 'Related AWS resources'. The main area is titled 'Functions (4/4)' and displays a table of four functions. Each function has a checkbox, a name, a description (all are '-'), a package type (all are 'Zip'), a runtime (Python 3.14 or 3.12), a type (all are 'Standard'), and a last modified time (all are '1 day ago'). The functions are 'ResizeImage', 'SendNotification', 'TriggerWorkflow', and 'DetectLabels'. At the top right of the console, the user's profile 'ANUSHA S' and the region 'Asia Pacific (Mumbai)' are visible.

	Function name	Description	Package type	Runtime	Type	Last modified
☒	ResizeImage	-	Zip	Python 3.14	Standard	1 day ago
☒	SendNotification	-	Zip	Python 3.12	Standard	1 day ago
☒	TriggerWorkflow	-	Zip	Python 3.12	Standard	1 day ago
☒	DetectLabels	-	Zip	Python 3.12	Standard	1 day ago

Step 3: Create SNS Topic

Created topic: image-processing-notification

Subscribed email endpoint

Confirmed email subscription

Step 4: Configure IAM Roles

Lambda execution roles created

Attached:

AWSLambdaBasicExecutionRole

AmazonS3ReadOnlyAccess

AWSStepFunctionsFullAccess

Added S3 invoke permission to trigger Lambda

Step 5: Create Step Function

Selected Standard workflow

Added states:

ResizeImage

DetectLabels

SendNotification

Configured JSONPath for input/output

Created execution role automatically

Execution: 8ba370da-34c5-4039-b068-5082efcdb2f5

Step Functions

- Dashboard
- State machines
- Execution inspector
- Activities

▼ **Developer resources**

- Online learning workshop
- Local Development
- Data flow simulator
- Feature spotlight
- Documentation

Join our feedback panel

Free virtual workshops

Execution details:

- Execution status:** Succeeded
- Execution type:** Standard
- Execution ARN:** arn:aws:states:ap-south-1:921292479204:execution:imageProcessingWorkflow:8ba370da-34c5-4039-b068-5082efcdb2f5
- IAM role ARN:** arn:aws:iam:921292479204:role/service-role/StepFunctions-ImageProcessingWorkflow-role-o504vxs3i
- State transitions:** 5
- Start time:** May 20, 2026, 20:11:55.966 (UTC+05:30)
- End time:** May 20, 2026, 20:11:56.788 (UTC+05:30)
- Duration:** 00:00:00.822
- Alias:** -
- Version:** -

Graph view | Table view

Graph view:

```

graph TD
    Start((Start)) --> AWSLambdaInvoke[AWS Lambda: Invoke]
  
```

Step details: Choose a step to view its details.

Execution: 8ba370da-34c5-4039-b068-5082efcdb2f5

Step Functions

- Dashboard
- State machines
- Execution inspector
- Activities

▼ **Developer resources**

- Online learning workshop
- Local Development
- Data flow simulator
- Feature spotlight
- Documentation

Join our feedback panel

Free virtual workshops

Graph view | Table view

Graph view:

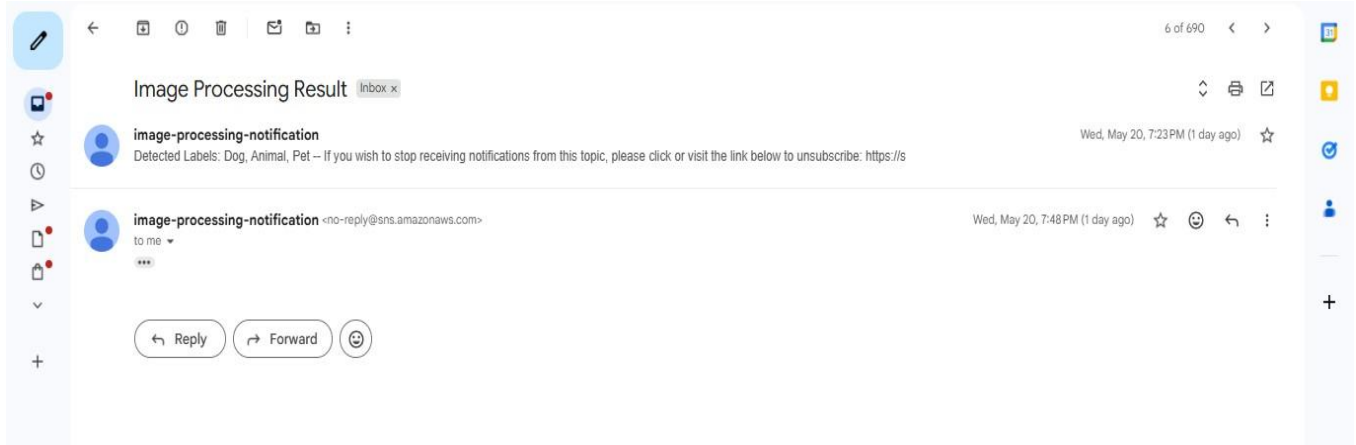
```

graph TD
    Start((Start)) --> AWSLambdaInvoke1[AWS Lambda: Invoke  
ResizeImage]
    AWSLambdaInvoke1 --> AWSLambdaInvoke2[AWS Lambda: Invoke  
DetectLabels]
    AWSLambdaInvoke2 --> AWSLambdaInvoke3[AWS Lambda: Invoke  
SendNotification]
    AWSLambdaInvoke3 --> End((End))
  
```

Step details: Choose a step to view its details.

Event view | State view

Legend: In progress, Failed, Caught error, Canceled, Succeeded



Confirmed successful execution

ImageProcessingWorkflow

Details

Arn: [arn:aws:states:ap-south-1:921292479204:stateMachine:ImageProcessingWorkflow](#)

IAM role ARN: [arn:aws:iam::921292479204:role/service-role/StepFunctions-ImageProcessingWorkflow-role-o504vxs3l](#)

Type: Standard

Status: Active

Creation date: May 20, 2026, 19:02:39 (UTC+05:30)

X-Ray tracing: Disabled

Executions (0/9)

Name	Status	Start Time (local)	End Time (local)	Duration	Version	Alias
8ba370da-34c5-4039-b068-5082efcd62f5	Succeeded	May 20, 2026, 20:11:55	May 20, 2026, 20:11:56	00:00:00.822	-	-
38d65f5c-8bec-4d0a-ba0e-54be2e9ed6d4	Succeeded	May 20, 2026, 19:44:47	May 20, 2026, 20:10:20	00:25:33.531	-	-
4f346869-5a11-40c4-a4d7-32a26c851c4f	Succeeded	May 20, 2026, 19:48:50	May 20, 2026, 19:48:50	00:00:00.717	-	-

Troubleshooting

Issue 1: Execution Not Starting Automatically

Method: Checked Lambda logs Cause: S3 event notification missing

Solution: Created event notification properly

Issue 2: Log Group Not Created

Method: Checked CloudWatch Cause: Lambda never triggered

Solution: Verified S3 trigger configuration

Issue 3: InvalidS3ObjectException

Method: Checked DetectLabels step Cause: Missing S3 read permission

Solution: Added AmazonS3ReadOnlyAccess policy

Issue 4: Step Function Not Starting

Method: Checked Trigger Lambda logs Cause: Missing start_execution logic

Solution: Updated Lambda code and environment variable

Issue 5: InvalidClientTokenId (SNS)

Method: Checked region mismatch

Cause: SNS and Lambda in different region Solution: Created SNS in same region

Conclusion

This project successfully demonstrates the implementation of a fully automated serverless image processing workflow using AWS services. The architecture is event-driven, scalable, secure, and does not require infrastructure management.

Key Achievements:

Designed and implemented serverless architecture

Integrated AI services (Amazon Rekognition)

Implemented workflow orchestration using Step Functions

Configured IAM roles and cross-service permissions

Built automated notification system

Performed debugging and troubleshooting of real-world errors

The project showcases strong understanding of cloud-native architecture, automation, and DevOps practices.